

Software Design Document

1. Introduzione

Il presente documento descrive l'architettura software del progetto Music Playlist Manager. Il sistema consiste in un'applicazione desktop Java per la creazione e la gestione di playlist musicali. L'applicazione permette all'utente di gestire una libreria di tracce, organizzarle in playlist e simulare la riproduzione dei brani attraverso comandi come play, pause e skip.

Le funzionalità principali previste sono:

- gestione delle tracce musicali;
- creazione, modifica, eliminazione e visualizzazione delle tracce;
- gestione delle playlist;
- aggiunta e rimozione di tracce da playlist;
- playback simulato di tracce e playlist;
- supporto alle modalità sequenziale, shuffle e loop;
- aggiornamento della UI durante il playback;
- possibile estensione con undo, statistiche, tag, filtri e smart playlist.

Il documento adotta il modello "4+1" di Kruchten, che descrive l'architettura attraverso cinque prospettive:

1. **Vista Logica**, per descrivere le astrazioni principali del sistema;
2. **Vista di Processo**, per descrivere il comportamento a runtime;
3. **Vista di Sviluppo**, per descrivere l'organizzazione del codice;
4. **Vista Fisica**, per descrivere il deployment;
5. **Scenari "+1"**, per validare l'architettura rispetto ai casi d'uso principali.

L'obiettivo del documento è fornire in modo sintetico e chiaro le principali decisioni di design, i componenti ad alto livello, le interfacce tra i moduli, i flussi di dati, la strategia di gestione degli errori e i pattern architetturali e di design adottati.

2. Vista Logica

La Vista Logica descrive la struttura funzionale del sistema e le principali astrazioni del dominio applicativo. In questa vista vengono identificati i moduli logici, le classi principali, le loro responsabilità e le relazioni fondamentali tra gli oggetti.

Nel nostro caso, il dominio ruota attorno a tre concetti principali:

- la **libreria musicale**, composta da tracce;
- le **playlist**, che organizzano le tracce;

- il **player simulato**, che gestisce lo stato della riproduzione.

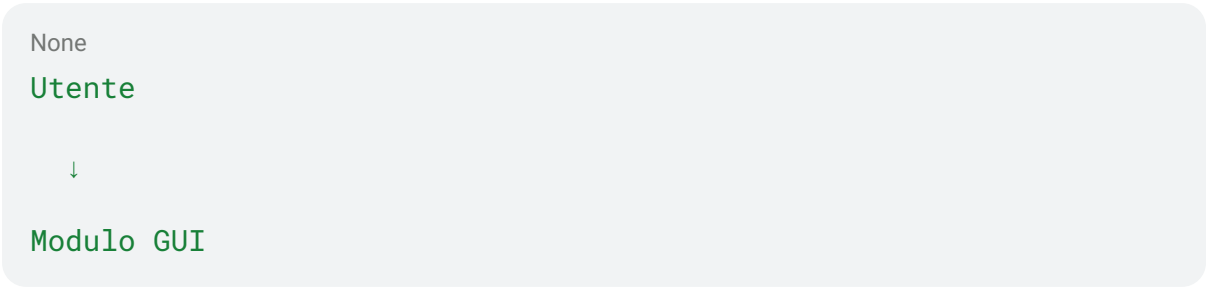
2.1 Componenti logici principali

A livello logico, il sistema può essere suddiviso nei seguenti componenti principali.

Componente	Responsabilità
Modulo GUI	Mostra tracce, playlist e stato del player; raccoglie gli input dell'utente.
Modulo Catalogo	Gestisce la libreria delle tracce musicali.
Modulo Playlist	Gestisce creazione, visualizzazione e modifica delle playlist.
Modulo Player	Gestisce il playback simulato, lo stato del player e la traccia corrente.
Modulo Repository	Astrae la memorizzazione e il recupero di tracce e playlist.
Modulo Error Handling	Gestisce validazioni ed eccezioni applicative.

Il sistema segue una logica a responsabilità separate. La GUI non deve contenere direttamente la logica di business; essa comunica con i controller, che a loro volta invocano i servizi applicativi.

Schema logico di alto livello:



```
↓  
Controller  
  
↓  
Service Layer  
  
↓  
Domain Model / Repository
```

2.2 Classi principali del dominio

Le principali classi del dominio sono le seguenti.

Track

Rappresenta una singola traccia musicale.

Attributi principali:

```
None  
- id  
  
- title  
  
- author  
  
- durationInSeconds  
  
- genre  
  
- year  
  
- tags
```

La traccia è l'entità base dell'applicazione. Ogni traccia contiene titolo, autore, durata, genere e anno di pubblicazione. In estensioni successive può essere arricchita con tag visivi come favourite, explicit e new release.

Playlist

Rappresenta una raccolta ordinata di tracce.

Attributi principali:

None

- `id`
- `name`
- `tracks`

Operazioni principali:

None

- + `addTrack(track)`
- + `removeTrack(track)`
- + `getTracks()`
- + `reorderTrack(fromIndex, toIndex)`

Una playlist contiene riferimenti a tracce presenti nella libreria. Rimuovere una traccia da una playlist non significa eliminarla dalla libreria generale.

PlaybackSession

Rappresenta lo stato corrente del playback simulato.

Attributi principali:

None

- `currentTrack`
- `currentPlaylist`
- `state`
- `mode`

Operazioni principali:

None

- + `playTrack(track)`

```
+ playPlaylist(playlist)
+ pause()
+ skip()
+ stop()
```

Questa classe separa la gestione dello stato del player dalla GUI. L'interfaccia mostra lo stato, ma non lo gestisce direttamente.

PlaybackState

Rappresenta gli stati possibili del player.

Valori principali:

```
None
STOPPED
PLAYING
PAUSED
```

L'uso di una enumerazione evita di rappresentare lo stato con stringhe libere, riducendo il rischio di errori.

PlaybackMode

Rappresenta la strategia con cui viene scelta la prossima traccia.

Implementazioni previste:

```
None
SequentialPlaybackMode
ShufflePlaybackMode
LoopPlaybackMode
```

Questa astrazione consente di gestire modalità diverse di playback senza concentrare tutta la logica dentro un unico metodo pieno di condizioni.

TrackTag

Rappresenta eventuali tag visivi associabili alle tracce.

Valori previsti:

None

FAVOURITE

EXPLICIT

NEW_RELEASE

Questa funzionalità è prevista tra quelle a priorità media, quindi la considereremo come estensione futura.

2.3 Relazioni tra le classi

Le relazioni principali sono:

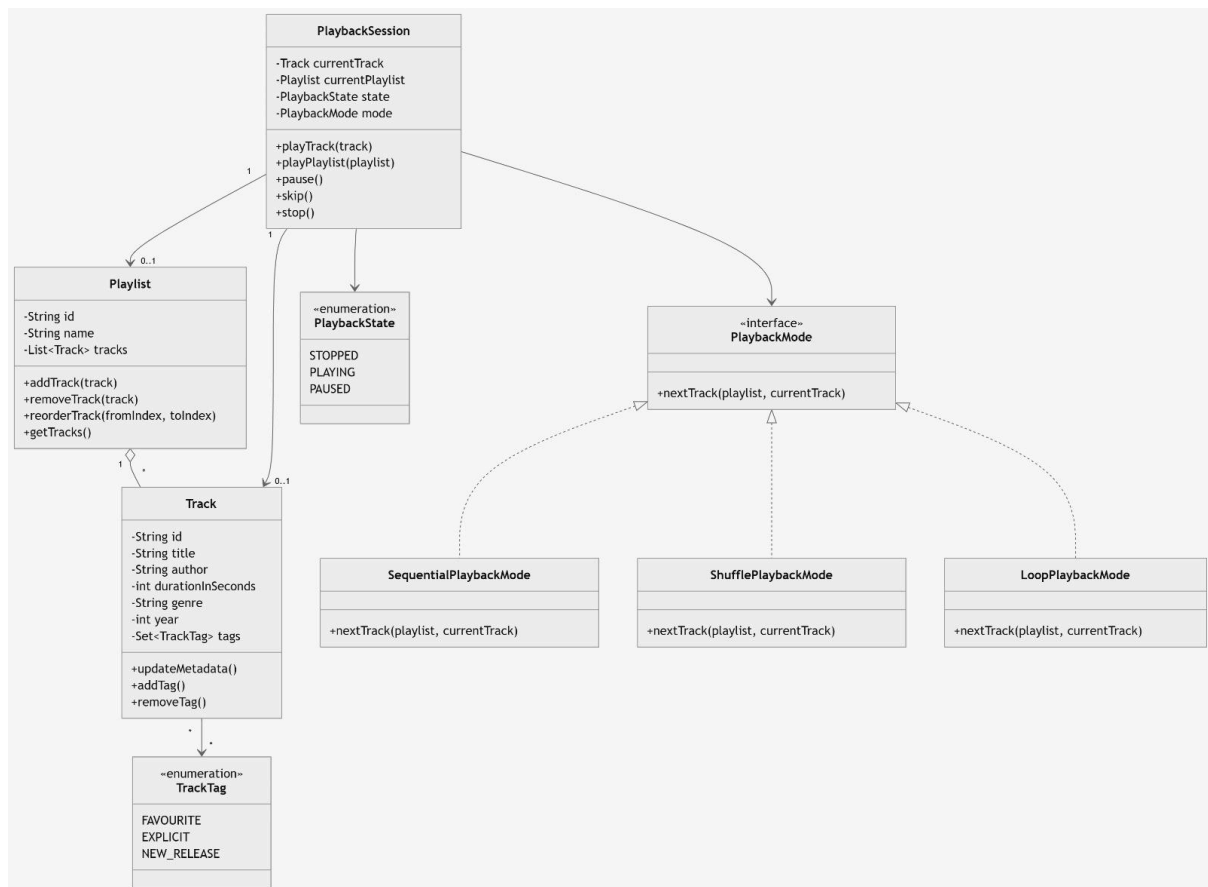
- una `Playlist` contiene zero o più `Track`;
- una `Track` può appartenere a più playlist;
- una `Track` può avere zero o più `TrackTag`;
- una `PlaybackSession` mantiene il riferimento alla traccia corrente;
- una `PlaybackSession` può mantenere il riferimento alla playlist corrente;
- una `PlaybackSession` possiede un `PlaybackState`;
- una `PlaybackSession` usa un `PlaybackMode`;
- `SequentialPlaybackMode`, `ShufflePlaybackMode` e `LoopPlaybackMode` implementano l'interfaccia `PlaybackMode`.

Questa struttura favorisce la separazione di concetti come:

- dato musicale;
- organizzazione dei brani;
- stato del player;
- strategia di riproduzione.

2.4 Diagramma delle classi

2.5 Pattern rilevanti nella vista logica



Nella vista logica emergono soprattutto due pattern:

- Il primo è lo **Strategy Pattern**, applicato alla gestione delle modalità di playback. Il comportamento di scelta della prossima traccia cambia a seconda della modalità attiva: sequenziale, shuffle o loop. Incapsulare questi comportamenti in classi separate evita di concentrare tutto nel **PlaybackService**.
- Il secondo è il **Repository Pattern**. Il dominio e i servizi non devono dipendere dal modo concreto in cui tracce e playlist vengono salvate. Nella prima versione i dati possono essere mantenuti in memoria, ma in futuro sarà possibile introdurre persistenza locale senza modificare la logica applicativa.

3. Vista di Processo

La Vista di Processo descrive il comportamento del sistema a runtime. In particolare, mostra come i componenti collaborano durante l'esecuzione, come fluiscono i dati tra i moduli, come viene gestito il playback simulato e come vengono trattati gli errori.

Essendo una desktop application, non prevede distribuzione del carico su più nodi. Tuttavia, è comunque importante descrivere il flusso degli eventi e la sincronizzazione tra stato interno e interfaccia grafica.

3.1 Comportamento runtime generale

Il comportamento generale del sistema è **event-driven**.

L'utente interagisce con la GUI, ad esempio cliccando su "Create Track", "Add to Playlist", "Play" o "Skip". La View raccoglie l'evento e lo inoltra al Controller. Il Controller invoca il Service appropriato. Il Service applica la logica applicativa, valida i dati, aggiorna il modello di dominio e usa il Repository se è necessario salvare o recuperare informazioni.

Flusso generale:

None

Utente → View → Controller → Service → Domain / Repository

Il risultato dell'operazione segue il percorso inverso:

None

Service → Controller → View → Utente

Questa struttura rispetta la separazione delle responsabilità e permette di testare la logica applicativa senza dipendere dalla GUI.

3.2 Flusso dei dati e interazioni

Un esempio rappresentativo è la creazione di una traccia.

1. L'utente inserisce titolo, autore, durata, genere e anno.
2. La View raccoglie i dati.
3. Il Controller costruisce una richiesta verso il **TrackService**.
4. Il **TrackService** valida i dati.
5. Se i dati sono validi, viene creato un oggetto **Track**.
6. Il **TrackRepository** salva la traccia.
7. La View viene aggiornata con la nuova lista delle tracce.

Flusso sintetico:

None

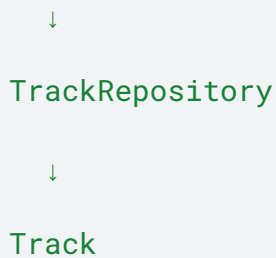
TrackView

↓

TrackController

↓

TrackService



Lo stesso principio vale per playlist e playback.

Le informazioni non devono passare direttamente dalla GUI al repository, perché questo favorirebbe un alto accoppiamento tra interfaccia e gestione dati.

3.3 Gestione del playback simulato

Il playback viene gestito dal modulo **Player**.

Il sistema mantiene una **PlaybackSession**, che contiene:

- traccia corrente;
- playlist corrente;
- stato corrente del player;
- modalità di playback attiva.

Quando l'utente avvia il playback di una playlist, il **PlaybackService** seleziona la traccia da riprodurre attraverso la strategia di playback attiva.

Esempio:

None
PlaybackService → **PlaybackSession** → **PlaybackMode**

La modalità di playback determina il comportamento:

Modalità	Comportamento
Sequential	Le tracce vengono riprodotte nell'ordine della playlist.

Shuffle	La prossima traccia viene scelta casualmente.
Loop	Alla fine della playlist, la riproduzione ricomincia.

Il playback è simulato, quindi il sistema non deve necessariamente gestire audio reale. Deve però aggiornare correttamente lo stato del player e la traccia corrente.

3.4 Aggiornamento della UI

Il sistema prevede un meccanismo di notifica dei cambiamenti di stato. Quando il playback cambia stato, ad esempio da **PLAYING** a **PAUSED**, oppure quando cambia la traccia corrente, la UI deve essere aggiornata.

Il flusso previsto è:

```

None
PlaybackService / PlaybackSession

  ↓ notifica evento

PlayerController / PlayerView

  ↓

Aggiornamento UI

```

Questa scelta è coerente con l'**Observer Pattern**, perché evita che la GUI debba interrogare continuamente il player. È il componente che gestisce lo stato a notificare i componenti interessati.

3.5 Gestione degli errori a runtime

Durante l'esecuzione possono verificarsi errori o input non validi. Alcuni esempi sono:

- titolo della traccia vuoto;
- autore vuoto;
- durata negativa o nulla;
- anno non valido;
- nome playlist vuoto;
- tentativo di riprodurre una playlist vuota;
- tentativo di fare skip senza una traccia successiva;
- tentativo di rimuovere da una playlist una traccia non presente.

La gestione degli errori segue questo flusso:

None

Service → eccezione applicativa → Controller → messaggio utente nella View

Le eccezioni applicative previste sono:

None

InvalidTrackException

InvalidPlaylistException

PlaybackException

Il Service è responsabile della validazione e della generazione dell'errore.

Il Controller intercetta l'errore e lo converte in un messaggio comprensibile per l'utente.

In questo modo, dettagli tecnici interni non vengono mostrati direttamente nella GUI.

3.6 Diagrammi di sequenza

Creazione di una traccia

None

sequenceDiagram

actor User as Utente

participant View as TrackView

participant Controller as TrackController

participant Service as TrackService

participant Repo as TrackRepository

User->>View: Inserisce dati traccia

User->>View: Conferma creazione

View->>Controller: createTrack(inputData)

```
Controller->>Service: createTrack(title, author, duration, genre,
year)
```

```
Service->>Service: valida input
```

```
alt dati validi
```

```
    Service->>Repo: save(track)
```

```
    Repo-->>Service: traccia salvata
```

```
    Service-->>Controller: traccia creata
```

```
    Controller-->>View: aggiorna lista tracce
```

```
    View-->>User: mostra nuova traccia
```

```
else dati non validi
```

```
    Service-->>Controller: InvalidTrackException
```

```
    Controller-->>View: mostra messaggio errore
```

```
    View-->>User: visualizza errore
```

```
end
```

Avvio del playback

```
None
```

```
sequenceDiagram
```

```
actor User as Utente
```

```
participant View as PlayerView
```

```
participant Controller as PlayerController
```

```
participant Service as PlaybackService
```

```
participant Session as PlaybackSession
```

```
participant Mode as PlaybackMode

User->>View: Seleziona playlist
User->>View: Clicca Play
View->>Controller: playPlaylist(playlistId)
Controller->>Service: playPlaylist(playlistId)
Service->>Session: startPlaylist(playlist)
Session->>Mode: nextTrack(playlist, currentTrack)
Mode-->>Session: prossima traccia
Session->>Session: state = PLAYING
Session-->>Service: stato aggiornato
Service-->>Controller: currentTrack + state
Controller-->>View: aggiorna UI
View-->>User: mostra traccia corrente
```

Skip della traccia corrente

```
None

sequenceDiagram
    actor User as Utente
    participant View as PlayerView
    participant Controller as PlayerController
    participant Service as PlaybackService
    participant Session as PlaybackSession
    participant Mode as PlaybackMode
```

```
User->>View: Clicca Skip
View->>Controller: skipTrack()
Controller->>Service: skip()
Service->>Session: skip()
Session->>Mode: nextTrack(currentPlaylist, currentTrack)
Mode-->>Session: prossima traccia
Session->>Session: aggiorna currentTrack
Session-->>Service: nuova traccia corrente
Service-->>Controller: currentTrack + state
Controller-->>View: aggiorna UI
View-->>User: mostra nuova traccia
```

4. Vista di Sviluppo

La Vista di Sviluppo descrive come il software viene organizzato nel codice sorgente, rendendo esplicita la struttura dei package, dei moduli fisici, delle librerie e dei test.

4.1 Organizzazione dei package

La struttura proposta dei package Java è la seguente:

```
None
it.unisa.musicplaylistmanager

|
|— app
|
```

```
├─ ui
|   ├─ view
|   └─ controller
|
├─ service
|
├─ domain
|   └─ playback
|
├─ repository
|
├─ exception
|
└─ test
```

Con questa organizzazione abbiamo cercato di mirare ad un'architettura a livelli e permette di separare la presentazione, la logica applicativa, il dominio e l'accesso ai dati.

4.2 Moduli fisici del codice

I principali moduli fisici sono:

Package	Responsabilità
app	Avvio dell'applicazione.

<code>ui.view</code>	Componenti grafici e schermate.
<code>ui.controller</code>	Gestione degli eventi della GUI.
<code>service</code>	Logica applicativa.
<code>domain</code>	Entità principali del dominio.
<code>domain.playback</code>	Strategie di playback.
<code>repository</code>	Interfacce e implementazioni per la gestione dei dati.
<code>exception</code>	Eccezioni specifiche dell'applicazione.
<code>test</code>	Test unitari.

4.3 Dipendenze tra package

Le dipendenze seguono una direzione precisa:

None

`ui.view → ui.controller → service → domain`

↓

`repository`

Regole principali:

- la View non accede direttamente al Repository;
- il Controller non contiene logica di business complessa;
- il Service coordina Domain e Repository;

- il Domain non dipende dalla GUI;
- il Repository non conosce la View;
- le eccezioni applicative vengono usate per comunicare errori controllati.

Abbiamo adottato questa struttura in quanto riduce l'accoppiamento e supporta il principio di separazione delle responsabilità.

4.4 Librerie e strumenti di build

Gli strumenti previsti sono:

Strumento	Uso
Java	Linguaggio di sviluppo dell'applicazione.
GitHub	Repository del progetto e versionamento.
JUnit	Test unitari.
Trello	Tracciamento attività, sprint backlog e task board.
Maven	Gestione build e dipendenze, da confermare dal team.

4.5 Organizzazione dei test

I test saranno collocati in una struttura separata dal codice principale.

Esempio:

None

`src/main/java`

`src/test/java`

I test dovrebbero concentrarsi principalmente sulla business logic:

- validazione delle tracce;
- creazione delle playlist;
- aggiunta e rimozione di tracce;
- comportamento delle modalità di playback;
- gestione degli errori applicativi.

Esempi di classi di test:

None

TrackServiceTest

PlaylistServiceTest

PlaybackServiceTest

SequentialPlaybackModeTest

La GUI può essere verificata con test manuali o, se il team lo ritiene opportuno, con test specifici in una fase successiva. Per la valutazione iniziale è importante che almeno le classi di business logic siano testabili in modo automatico.

5. Vista Fisica

La Vista Fisica descrive come il software viene distribuito sull'infrastruttura hardware e nel nostro caso, essendo un'applicazione desktop stand-alone, il deployment non sarà molto complesso.

5.1 Ambiente di esecuzione

L'applicazione viene eseguita localmente sul computer dell'utente.

Ambiente previsto:

None

PC / Laptop dell'utente

Sistema operativo desktop

Java Virtual Machine

5.3 Gestione iniziale dei dati

Nella prima versione, i dati possono essere mantenuti in memoria attraverso repository in-memory.

Esempi:

None

InMemoryTrackRepository

InMemoryPlaylistRepository

Questa scelta semplifica lo sviluppo iniziale e permette al team di concentrarsi sulle funzionalità fondamentali: gestione tracce, gestione playlist e playback simulato.

In futuro, grazie alle interfacce repository, sarà possibile introdurre forme di persistenza senza modificare radicalmente la logica applicativa.

5.4 Possibili estensioni future

Possibili evoluzioni del deployment sono:

- persistenza su file locale;
- database embedded;
- esportazione/importazione di playlist;
- sincronizzazione cloud;
- gestione multiutente.

6. Scenari “+1”

Gli scenari “+1” servono a validare le altre viste architetturali. Mostrano come la Vista Logica, la Vista di Processo, la Vista di Sviluppo e la Vista Fisica cooperano per realizzare casi d'uso concreti.

6.1 Scenario: creazione di una traccia

L'utente inserisce titolo, autore, durata, genere e anno di una nuova traccia.

Flusso:

None

Utente → TrackView → TrackController → TrackService → TrackRepository

Componenti coinvolti:

- TrackView;
- TrackController;
- TrackService;
- Track;
- TrackRepository.

Validazione:

- titolo non vuoto;
- autore non vuoto;
- durata positiva;
- anno valido.

Pattern coinvolti:

- MVC;
- architettura a livelli;
- Repository Pattern.

6.2 Scenario: creazione di una playlist

L'utente inserisce il nome di una nuova playlist.

Flusso:

None

Utente → PlaylistView → PlaylistController → PlaylistService → PlaylistRepository

Componenti coinvolti:

- PlaylistView;
- PlaylistController;
- PlaylistService;
- Playlist;
- PlaylistRepository.

Validazione:

- nome playlist non vuoto;
- playlist creata inizialmente senza tracce.

Pattern coinvolti:

- MVC;
- architettura a livelli;
- Repository Pattern.

6.3 Scenario: aggiunta traccia a playlist

L'utente seleziona una traccia dalla libreria e una playlist di destinazione.

Flusso:

None

Utente → PlaylistView → PlaylistController → PlaylistService

Il **PlaylistService** recupera traccia e playlist, poi aggiorna la playlist.

La traccia non viene rimossa dalla libreria generale: viene soltanto associata alla playlist.

Pattern coinvolti:

- MVC;
- Repository Pattern;
- separazione delle responsabilità tra catalogo e playlist.

6.4 Scenario: playback simulato

L'utente seleziona una playlist e clicca su Play.

Flusso:

None

Utente → PlayerView → PlayerController → PlaybackService → PlaybackSession → PlaybackMode

Il sistema:

1. seleziona la playlist corrente;
2. imposta lo stato del player a **PLAYING**;
3. sceglie la traccia corrente in base alla modalità attiva;
4. aggiorna la UI.

Pattern coinvolti:

- MVC;
- Strategy Pattern;
- Observer Pattern.

6.5 Scenario: gestione input non valido

L'utente prova a creare una traccia con un anno non valido o una durata negativa.

Flusso:

None

`TrackView → TrackController → TrackService`

Il `TrackService` valida i dati e genera una `InvalidTrackException`.

Il controller intercetta l'eccezione e chiede alla View di mostrare un messaggio comprensibile.

Esempio:

None

`"Anno non valido."`

`"La durata della traccia deve essere maggiore di zero."`

Pattern e principi coinvolti:

- separazione tra validazione e presentazione;
- gestione centralizzata degli errori;
- Service Layer.

6.6 Validazione delle viste attraverso gli scenari

Scenario	Vista logica	Vista processo	Vista sviluppo	Vista fisica
Creazione traccia	<code>Track</code>	flusso View-Service-R epository	package <code>domain</code> , <code>service</code> , <code>repository</code>	esecuzione e locale

Creazione playlist	<code>Playlist</code>	validazione e salvataggio	<code>PlaylistService</code>	esecuzione locale
Aggiunta traccia	relazione Track-Playlist	aggiornamento stato playlist	<code>PlaylistController</code> e <code>PlaylistService</code>	dati locali
Playback	<code>PlaybackSession</code> e <code>PlaybackMode</code>	stato runtime del player	<code>domain.playback</code>	JVM locale
Input non valido	eccezioni dominio	errore runtime controllato	package <code>exception</code>	nessun impatto fisico

7. Decisioni architetturali principali

Decisione	Motivazione
Applicazione desktop Java	Richiesta dalla traccia del progetto.
Architettura a livelli	Separa GUI, logica applicativa, dominio e dati.
MVC	Disaccoppia interfaccia grafica e logica applicativa.
Repository Pattern	Astrae la gestione dei dati.
Storage iniziale in memoria	Riduce la complessità iniziale.

Strategy Pattern per playback	Permette di gestire modalità diverse senza modificare il player.
Observer Pattern per UI update	Aggiorna la UI quando cambia lo stato del playback.
Command Pattern per undo futuro	Supporta operazioni annullabili.
JUnit per test	Permette di verificare automaticamente la business logic.
Trello e GitHub	Supportano tracciamento Scrum e versionamento.

8. Design Pattern utilizzati

Architettura a livelli

Il sistema viene diviso in livelli:

None

View → Controller → Service → Domain / Repository

Questa scelta mira a migliorare la separazione delle responsabilità, manutenibilità e testabilità.

Model-View-Controller

MVC viene usato per la gestione della GUI.

- **Model:** `Track`, `Playlist`, `PlaybackSession`;
- **View:** schermate dell'applicazione;
- **Controller:** classi che gestiscono gli input dell'utente.

MVC evita che la logica di business venga inserita direttamente nella GUI.

Repository Pattern

Il Repository Pattern isola il Service Layer dai dettagli di memorizzazione.

None

`TrackService → TrackRepository → InMemoryTrackRepository`

Il vantaggio è che in futuro si potrà passare da storage in memoria a persistenza su file o database senza modificare la logica applicativa.

Strategy Pattern

Strategy viene usato per le modalità di playback.

None

`PlaybackMode`

└─ `SequentialPlaybackMode`

└─ `ShufflePlaybackMode`

└─ `LoopPlaybackMode`

Ogni modalità incapsula il proprio algoritmo per scegliere la prossima traccia.

Observer Pattern

Observer viene usato per aggiornare la UI quando cambia lo stato del playback.

None

`PlaybackSession / PlaybackService → notifica → PlayerView`

Il vantaggio è evitare polling continuo e ridurre l'accoppiamento tra player e GUI.

Command Pattern

Command è previsto per supportare l'undo.

None

`AddTrackToPlaylistCommand`

```
RemoveTrackFromPlaylistCommand
```

Ogni comando può esporre:

```
None  
execute()  
undo()
```

Questo permette di annullare operazioni come aggiunta o rimozione di tracce da playlist.

9. Strategia di gestione degli errori

La strategia di gestione degli errori si basa su tre livelli:

1. validazione nel Service Layer;
2. eccezioni applicative specifiche;
3. messaggi utente gestiti dalla View.

Errori previsti

Errore	Gestione
Titolo vuoto	<code>InvalidTrackException</code>
Autore vuoto	<code>InvalidTrackException</code>
Durata non valida	<code>InvalidTrackException</code>
Anno non valido	<code>InvalidTrackException</code>
Nome playlist vuoto	<code>InvalidPlaylistException</code>

Playlist vuota in playback	<code>PlaybackException</code>
Skip non possibile	<code>PlaybackException</code>
Rimozione traccia assente	errore controllato o operazione ignorata in modo sicuro

Flusso di gestione errore

None

`Service`

↓ `lancia eccezione`

`Controller`

↓ `intercetta eccezione`

`View`

↓ `mostra messaggio`

`Utente`

Esempio:

None

`InvalidTrackException`

→ `"La durata della traccia deve essere maggiore di zero."`

Abbiamo adottato questa scelta per evitare di mostrare dettagli tecnici all'utente e mantenere separata la logica di validazione dalla logica di presentazione.

10. "Definition of Done"

Una User Story si considera DONE solo quando soddisfa TUTTI i criteri obbligatori (✓) elencati di seguito, oltre ai propri Acceptance Criteria specifici. I criteri indicati da questo simbolo (○) si considerano opzionali.

Codice

1. **Compilazione pulita:** Il codice compila senza errori e i warning vengono gestiti in maniera appropriata ✓
2. **Naming conventions:** Le naming conventions Java sono rispettate ✓
3. **Version control:** Il codice è stato caricato (committato e pushato) sul branch principale del repository condiviso ✓
4. **Commenti significativi:** I commenti descrivono il "perché" delle scelte non ovvie, non ridondano il codice ✓

Design

5. **Alta coesione:** Alta coesione tra responsabilità delle classi ✓
6. **Basso accoppiamento:** Basso accoppiamento tra classi/package ✓
7. **Astrazione:** Uso di interfacce o classi astratte ove necessario ○
8. **Principio DRY:** Assenza di duplicazioni (Don't Repeat Yourself) ✓
9. **Rispetto dell'architettura:** Le modifiche rispettano l'architettura definita e non introducono violazioni dei layer/package ✓

Test

10. **Verifica dei criteri:** Ogni Acceptance Criterion della User Story è stato verificato con successo (tramite test automatizzati JUnit per la logica di backend e test manuali per l'interfaccia UI) ✓
11. **Copertura scenari:** Copertura dei Casi Limite essenziali: i test includono il percorso ideale e i principali scenari di errore previsti dalla card ✓
12. **Test superati:** Passaggio dei Test: tutti i test passano prima dell'integrazione nel branch principale ✓

Processo SCRUM

13. **Integrazione codice:** Codice integrato tramite merge/review nel repository GitHub ✓
14. **Code Review:** Revisione del codice effettuata da almeno un altro membro ✓
15. **Aggiornamento board:** Task spostato in Done su Trello ✓
16. **Artefatti Scrum:** Aggiornamento degli artefatti Scrum quando necessario ○

Interfaccia Utente

17. **Aggiornamento dinamico UI:** L'interfaccia aggiorna correttamente lo stato senza riavvio ✓
18. **Feedback visivo:** Messaggi di errore comprensibili e visibili ✓